Audspect BAS — Performance Tuning Guide

Classification: Internal — Audspect Engineering / Confidential
Platform Version: v1.7.5

Overview

This guide covers performance optimization for medium-to-large deployments (50+ agents, daily runs). For standard deployments (<50 agents), the default configuration is sufficient.

Bottlenecks and Their Sources

Bottleneck	Source	Impact
Database connection saturation	Many concurrent runs returning results	Run result ingestion delays
PDF generation slow	Chromium memory limit	Report download latency
WebSocket fan-out	Many browser sessions + frequent broadcasts	Dashboard lag

Disk I/O	Large `scenario_runs.results` JSONB payloads on wide-scan runs	Slow run detail loads
Agent heartbeat storm	All agents reconnecting simultaneously after outage	Temporary CPU spike

PostgreSQL Tuning

Connection pool

The orchestrator uses a connection pool. Default max connections: 25. For deployments with 50+ concurrent runs:

In `docker-compose.yml` postgres service:

```
command:
  - postgres
  - -c
  - max_connections=150
  - -c
  - shared_buffers=1GB
  - -c
  - effective_cache_size=3GB
  - -c
  - work_mem=16MB
  - -c
  - maintenance_work_mem=256MB
  - -c
  - checkpoint_completion_target=0.9
  - -c
  - wal_buffers=16MB
  - -c
  - default_statistics_target=100
```

Index health

Verify indexes are being used for common queries:

```
-- Check for sequential scans on large tables (should be
SELECT schemaname, tablename, seq_scan, idx_scan
FROM pg_stat_user_tables
WHERE seq_scan > 0 AND n_live_tup > 10000
ORDER BY seq_scan DESC;
```

VACUUM schedule

For deployments with many completed runs (high INSERT/UPDATE/DELETE volume):

```
-- Check autovacuum lag
SELECT schemaname, tablename, last_autovacuum, last_autoa
FROM pg_stat_user_tables
ORDER BY n_dead_tup DESC LIMIT 10;
```

If `n_dead_tup` is consistently high on `run_steps` or `run_results`, tune autovacuum:

```
ALTER TABLE run_steps SET (autovacuum_vacuum_scale_factor
ALTER TABLE run_steps SET (autovacuum_analyze_scale_facto
```

Orchestrator Tuning

Orchestrator environment variables (performance)

```
# Increase DB connection pool size (default: 25)
```

```
BAS_DB_MAX_CONNECTIONS=50

# Increase WebSocket write buffer (reduces blocking on sl
BAS_WS_WRITE_BUFFER=65536

# Increase result ingestion worker pool (default: 10 work
BAS_RESULT_WORKERS=20
```

These are not yet exposed in v1.7.3 — add when needed by extending `config.go`.

Reducing WebSocket fan-out

If many browser sessions are open and broadcasts are frequent (during large campaign runs), add debouncing to `renderAgentRows()` in the dashboard JS:

```
var _renderAgentRowsDebounce;
function scheduleRenderAgentRows() {
  clearTimeout(_renderAgentRowsDebounce);
  _renderAgentRowsDebounce = setTimeout(renderAgentRows,
}
// Replace direct renderAgentRows() calls in WS handlers
```

Chromium PDF Tuning

The compose service is named `chrome` (image `chromedp/headless-shell`), not `chromium`. Reports render on-demand from HTML at request time via this sidecar — there is no pre-generated PDF cache or blob store to manage; if the sidecar is unavailable, PDF generation falls back to a built-in (lower-fidelity) renderer rather than failing outright.

```
services:
  chrome:
```

```
    mem_limit: 2g
    cpus: '2'
```

PDF generation is synchronous (one PDF at a time per Chromium instance). `BAS_CHROMIUM_URLS`/multi-instance round-robin is not implemented today — this would need to be added if concurrent PDF generation at scale becomes a real bottleneck; don't assume it exists.

Agent Heartbeat Tuning

Default heartbeat interval: 30 seconds. For large fleets, stagger agent reconnects to avoid thundering herd after an outage:

The agent binary randomizes reconnect delay (0–30 seconds) by default. If agents are all deployed simultaneously and restart simultaneously (power event), expect a 30–60 second spike at the server. The orchestrator's heartbeat handler is designed to handle this without dropping connections.

For very large fleets (200+ agents), increase reconnect jitter:

```
agent config: reconnect_jitter_seconds = 60
```

Disk Management

Reports render on-demand from HTML at request time (see Chromium PDF Tuning above) — there is no growing PDF/HTML blob table to archive or clean up. `report_log` records only lightweight generation metadata (type, format, scope, timestamp), not file content, and doesn't need retention management.

Run History Retention

Per-step results live in `scenario_runs.results`, a JSONB column directly on the run row — there is no separate `run_steps` child table. For deployments with many agents running daily, this is the table that grows fastest:

```
-- Check size
SELECT pg_size_pretty(pg_total_relation_size('scenario_ru

-- Archive runs older than 1 year (keep metadata, drop th
UPDATE scenario_runs
SET results = '[]'
WHERE completed_at < NOW() - INTERVAL '1 year'
  AND results != '[]';

VACUUM ANALYZE scenario_runs;
```

This is destructive to the per-step detail (raw output, evidence) of archived runs — the run's score, status, and summary fields are untouched, only `results` is cleared. Confirm nothing still needs that raw detail (e.g. an open finding's evidence trail) before running this in production.

Sizing Reference

Fleet Size	Orchestrator CPU	Orchestrator RAM	PostgreSQL RAM	Disk/ year
≤10 agents, daily	2 vCPU	2 GB	1 GB	~10 GB

10–50 agents, daily	4 vCPU	4 GB	2 GB	~50 GB
50–200 agents, daily	8 vCPU	8 GB	4 GB	~200 GB
200+ agents, continuous	16 vCPU	16 GB	8 GB	Contact us

These figures assume:

- One full ART scan (300 steps) per agent per day
- PDF reports retained for 90 days
- No SharpHound (AP collection adds ~20% to estimates)

© Audspect Engineering — Internal / Confidential